

MINI-PROJECT_DSBDA

Name-Sahil Narke

Roll_No:13245

Batch_ B3

Topic:-Performing all data analytical techniques using "TITANIC"
Dataset

1]Data Wrangling

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import seaborn as sns
```

```
df = pd.read_csv(r"C:\Users\Dell\Desktop\titanic.csv")
```

```
df
```

	PassengerId	Name
Pclass \		
0	1	Braund, Mr. Owen Harris
3	2	Cumings, Mrs. John Bradley (Florence Briggs Th...
1		
1		Heikkinen, Miss. Laina
2	3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
3	4	Allen, Mr. William Henry
3	5	
1		
4		...
3	...	Montvila, Rev. Juozas
..	887	
...		Graham, Miss. Margaret Edith
886	888	Johnston, Miss. Catherine Helen "Carrie"
2		
887	889	Behr, Mr. Karl Howell
1	890	Dooley, Mr. Patrick
888		
3	891	
889		
1		
890		
3		

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
Embarked \							
0	male	22.0	1	1	A/5 21171	7.2500	NaN
S							
1	female	38.0	0	0	PC 17599	71.2833	C85
C	female	26.0	1	0	STON/O2. 3101282	7.9250	NaN
2							
S	female	35.0		0	113803	53.1000	C123
3							

```

S 4
S .. male 35.0 0 0 373450 8.0500 NaN
...
886 ...male 27.0 0 0 211536 13.0000 NaN
S female 19.0 0 0 112053 30.0000 B42
887 NaN 1 2 W./C. 6607 23.4500 NaN
S male 26.0 0 0 111369 30.0000 C148
888 male 32.0 0 0 370376 7.7500 NaN
S
S
889 female
C
890
Q

```

```

Survived
0 0
1 1
2 1
3 1
4 0
..
886 0
887 1
888 0
889 1
890 0

```

[891 rows x 12 columns]

```
df.head(5)
```

```

PassengerId Name
Pclass \
0 1 Braund, Mr. Owen Harris
3 2 Cumings, Mrs. John Bradley (Florence Briggs Th...
1 Heikkinen, Miss. Laina
2 3 Futrelle, Mrs. Jacques Heath (Lily May Peel)
3 4 Allen, Mr. William Henry
1 5
4
3

Sex Age SibSp Parch Ticket Fare Cabin
Embarked \
0 male 22.0 1 0 A/5 21171 7.2500 NaN
S

```

```

1   female 38.0    1    0          PC 17599   71.2833   C85
C   female 26.0    0    0   STON/O2. 3101282   7.9250  NaN
2   female 35.0    1    0          113803   53.1000  C123
S   male 35.0     0    0          373450    8.0500  NaN
3
S
4
S

```

```

Survived
0      0
1      1
2      1
3      1
4      0

```

df.tail(5)

```

      PassengerId      Name  Pclass
Sex \
886      887      Montvila, Rev. Juozas      2
male
887      888      Graham, Miss. Margaret Edith      1
female
888      889 Johnston, Miss. Catherine Helen "Carrie"      3
female
889      890      Behr, Mr. Karl Howell      1
male
890      891      Dooley, Mr. Patrick      3
male

```

```

      Age SibSp Parch      Ticket   Fare Cabin Embarked Survived
886  27.0     0     0      211536   13.00   NaN        S         0
887  19.0     0     0      112053   30.00   B42        S         1
888  NaN     1     2  W./C. 6607   23.45   NaN        S         0
889  26.0     0     0      111369   30.00  C148        C         1
890  32.0     0     0      370376    7.75   NaN        Q         0

```

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId 891 non-null   int64
1   Name        891 non-null   object
2   Pclass      891 non-null   int64
3   Sex         891 non-null   object
4   Age         714 non-null   float64

```



```

887      False      False      False      False      False      False      False      False      False
False      False      False      False      False      True      False      False      False
888      False      False      False      False      False      False      False      False      False
False      False      False      False      False      False      False      False      False
889
False
890
False

```

```

      Cabin Embarked Survived
0      True      False      False
1      False      False      False
2      True      False      False
3      False      False      False
4      True      False      False
..      ...      ...      ...
886     True      False      False
887     False      False      False
888     True      False      False
889     False      False      False
890     True      False      False

```

```
[891 rows x 12 columns]
```

```
df.notnull()
```

```

      PassengerId Name Pclass      Sex      Age  SibSp  Parch      Ticket
Fare \
0      True      True      True      True      True      True      True      True
1      True      True      True      True      True      True      True      True
2      True      True      True      True      True      True      True      True
3      True      True      True      True      True      True      True      True
4      True      ...      ...      ...      ...      ...      ...      ...
886     True      True      True      True      False      True      True      True
887     True      True      True      True      True      True      True      True
888     True      True      True      True      True      True      True      True
889
890
True

```

		Cabin	Embarked	Survived
0	1	False	True	True
2	3	True	True	True
4	..	False	True	True
886		True	True	True
887		False	True	True
888		...	...	...
889		False	True	True
890		True	True	True
		False	True	True
		True	True	True
		False	True	True

[891 rows x 12 columns]

2]Data Formatting and Normalization

```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
df = sns.load_dataset('titanic')
numeric_cols = df.select_dtypes(include='number').columns
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0.0	1.0	male	0.271174	0.125	0.0	0.014151	S
1	1.0	0.0	female	0.472229	0.125	0.0	0.139136	C
2	1.0	1.0	female	0.321438	0.000	0.0	0.015469	S
3	1.0	0.0	female	0.434531	0.125	0.0	0.103644	S
4	0.0		male	0.434531	0.000	0.0	0.015713	S
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								
31								
32								
33								
34								
35								
36								
37								
38								
39								
40								
41								
42								
43								
44								
45								
46								
47								
48								
49								
50								
51								
52								
53								
54								
55								
56								
57								
58								
59								
60								
61								
62								
63								
64								
65								
66								
67								
68								
69								
70								
71								
72								
73								
74								
75								
76								
77								
78								
79								
80								
81								
82								
83								
84								
85								
86								
87								
88								
89								
90								
91								
92								
93								
94								
95								
96								
97								
98								
99								

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

3]Numeric Variables for Outliers

We'll use IQR (Interquartile Range) method

```

# Define the numeric columns to visualize and clean
df = sns.load_dataset("titanic")
print(df.select_dtypes(include='number').columns)

Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'],
      dtype='object')

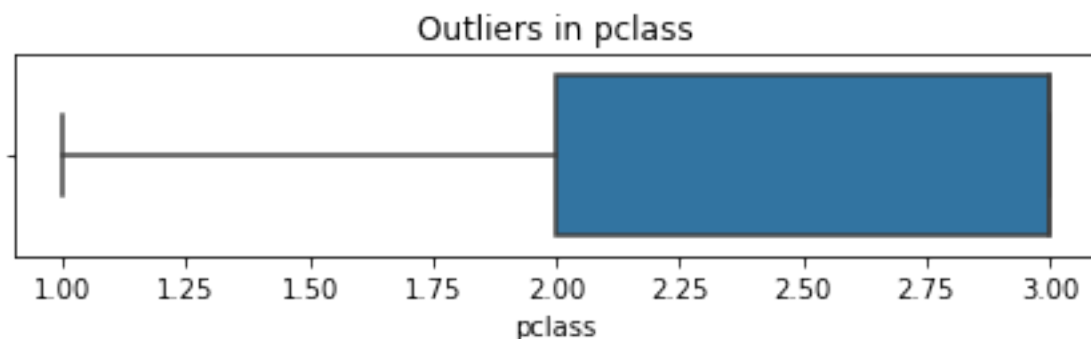
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

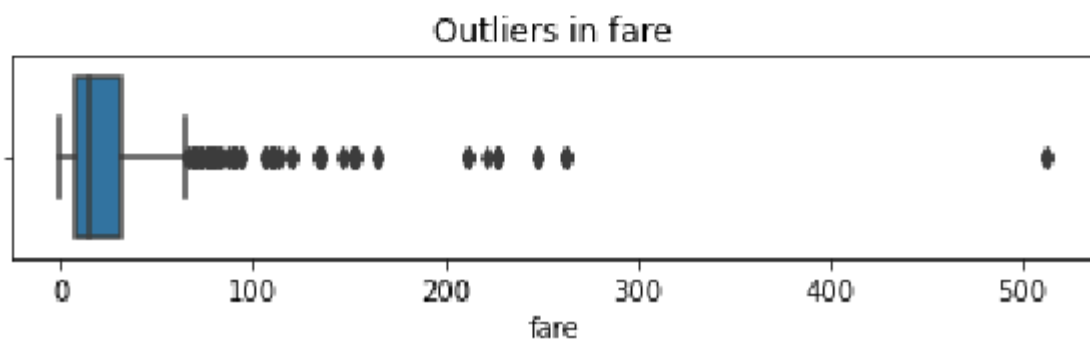
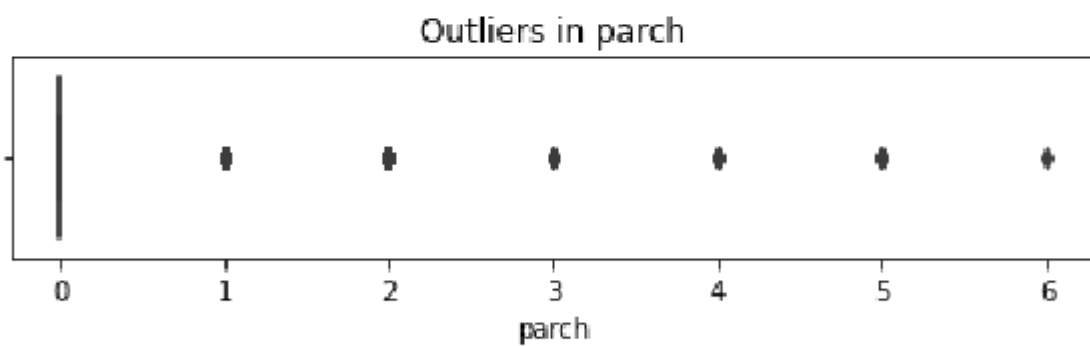
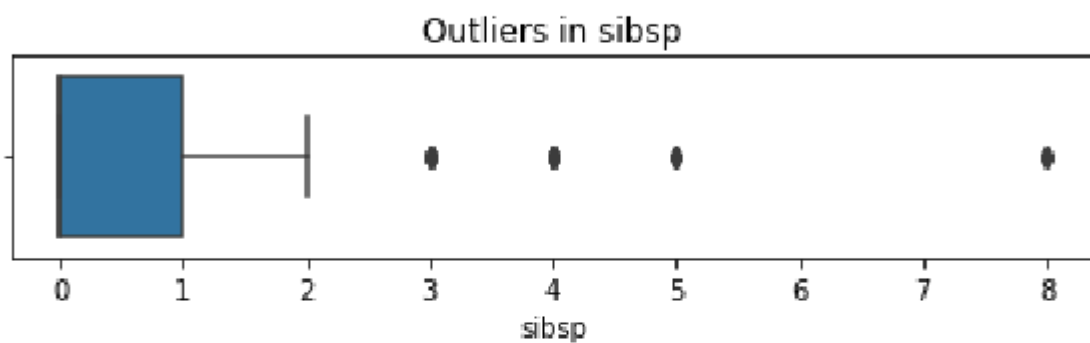
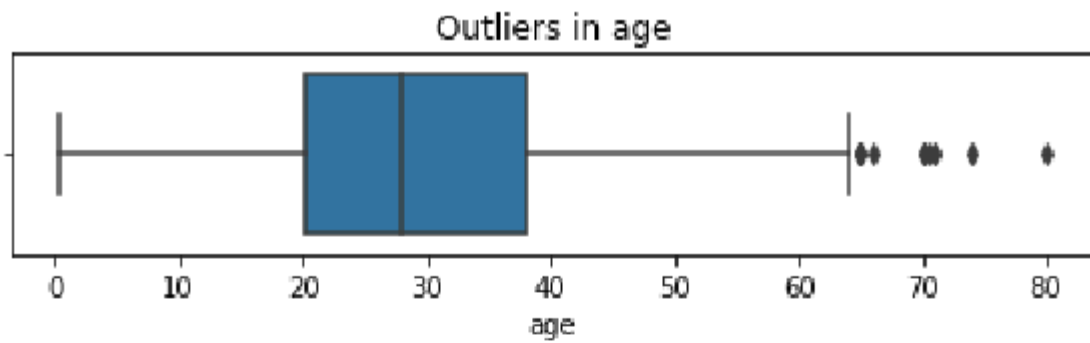
df = sns.load_dataset("titanic")
numeric_cols = ['pclass', 'age', 'sibsp', 'parch', 'fare', 'survived']
# --- Boxplots to visualize outliers ---
for col in numeric_cols:
    plt.figure(figsize=(6, 2))
    sns.boxplot(x=df[col])
    plt.title(f"Outliers in {col}")
    plt.tight_layout()
    plt.show()

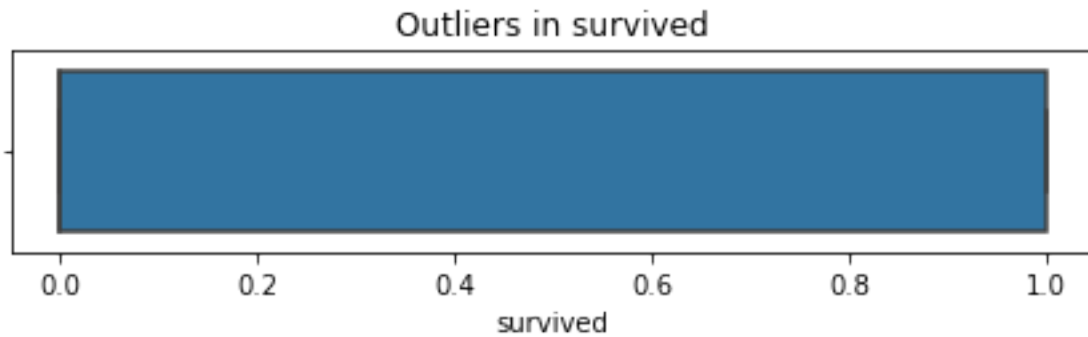
# --- Detect and handle outliers using the IQR method ---
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[col] = df[col].apply(lambda x: lower if x < lower else upper if
x > upper else x)

# Optional: Display the cleaned data
print(df.head())

```







	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \			male	22.0	1.0	0.0		S
0	0	3	1 female	38.0	1.0	0.0	7.2500	C
Third	1	1	3 female	26.0	0.0	0.0	65.6344	S
1	1		1 female	35.0	1.0	0.0	7.9250	S
First	0		male	35.0	0.0	0.0	53.1000	S
2							8.0500	
Third								
3								
First								
4		3						
Third								

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

5]Data Transformation

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Check skewness before transformation
print("Skewness before:", df['fare'].skew())

# Apply log transformation (log1p handles zeros)
df['fare_log'] = np.log1p(df['fare'])

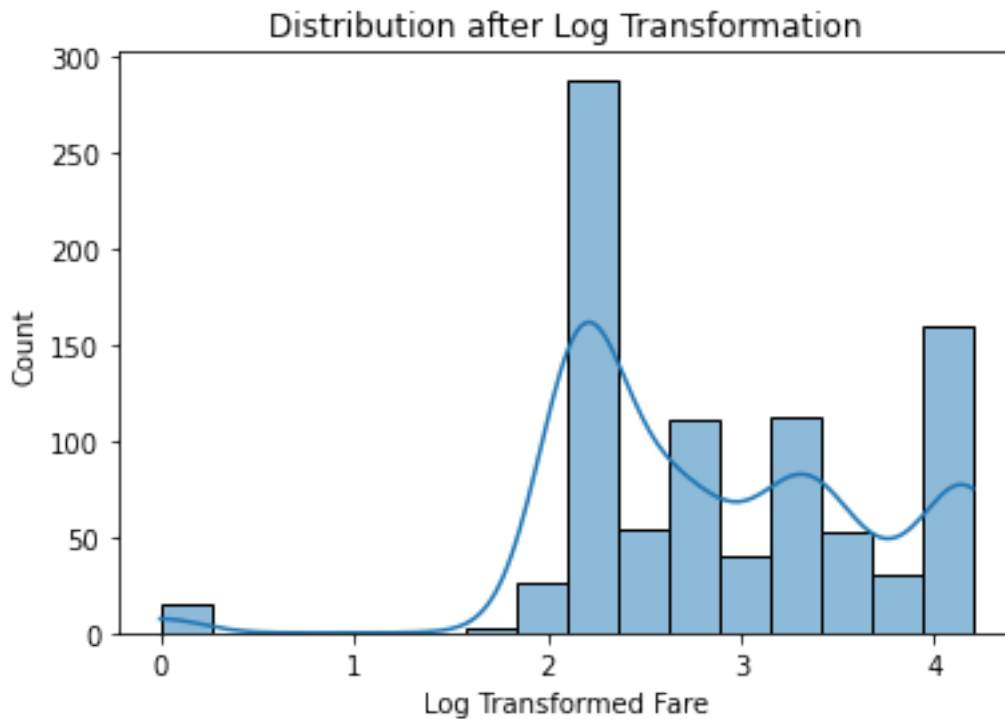
# Check skewness after transformation
print("Skewness after:", df['fare_log'].skew())

# Plot to visualize distribution
sns.histplot(df['fare_log'], kde=True)
plt.title("Distribution after Log Transformation")
```

```
plt.xlabel("Log Transformed Fare")
plt.show()
```

Skewness before: 1.082178592000912

Skewness after: -0.2378769041514196



6] Display basic statistical details (percentile, mean, standard deviation) for each species

```
sex_stats = df.groupby('sex').agg({
    'pclass': ['mean', 'std', 'min', 'max', 'median'],
    'survived': ['mean', 'std', 'min', 'max', 'median'],
    'age': ['mean', 'std', 'min', 'max', 'median'],
    'fare': ['mean', 'std', 'min', 'max', 'median']
})
```

```
# Display the stats for each species
print(sex_stats)
```

	pclass					survived				
	mean	std	min	max	median	mean	std	min	max	
female										
1.0	2.159236	0.85729	1	3	2.0	0.742038	0.438211	0	1	
male										
	2.389948	0.81358	1	3	3.0	0.188908	0.391775	0	1	

0.0

		age				fare	
\ std							
sex		mean	std	min	max	median	mean
female							
		27.915709	14.110146	0.75	63.0000	27.0	30.566906
	22.287475						
male		30.606198	14.357762	0.42	64.8125	29.0	20.498617
	18.505930						
		min	max		median		
sex							
female		6.75	65.6344			23.0	
male		0.00	65.6344			10.5	

7]Create a Linear Regression Model using Python/R

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
df = sns.load_dataset("titanic")

df = df[['fare', 'age', 'pclass', 'sibsp', 'parch']].dropna()

X = df[['age', 'pclass', 'sibsp', 'parch']]
y = df['fare']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=77)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse:.2f}")
print(f"R-squared: {r2:.2f}")
print("Model Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
Mean Squared Error: 1033.20
R-squared: 0.37
Model Coefficients: [ -0.17819748 -38.39576734
```

5.51780003

10.63774832]

Intercept: 119.6184039426405

[8] Logistic Regression on the Iris dataset using Python, and compute the Confusion Matrix along with evaluation metrics like:

True Positives (TP)

False Positives (FP)

True Negatives (TN)

False Negatives (FN)

Accuracy

Error rate

Precision

Recall

1. Import libraries

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import confusion_matrix, accuracy_score,  
precision_score, recall_score
```

2. Load Titanic dataset

```
df = sns.load_dataset('titanic')
```

3. Select relevant columns

```
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',  
'fare']]
```

4. Drop missing values

```
df = df.dropna()
```

5. Convert categorical variable 'sex' to numeric

```
df['sex'] = df['sex'].map({'male': 0, 'female': 1})
```

6. Define features and target

```
X = df[['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare']]
```

```
y = df['survived']
```

7. Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.3, random_state=42)
```

8. Train logistic regression model

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

9. Make predictions

```
y_pred = model.predict(X_test)
```

10. Confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
tn, fp, fn, tp = cm.ravel()
```

11. Metrics

```
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
```

12. Output results

```
print("Confusion Matrix:\n", cm)
print("\nMetrics:")
print(f"True Positives (TP): {tp}")
print(f"False Positives (FP): {fp}")
print(f"True Negatives (TN): {tn}")
print(f"False Negatives (FN): {fn}")
print(f"Accuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
```

Confusion Matrix:

```
[[107   19]
 [ 30   59]]
```

Metrics:

```
True Positives (TP): 59
False Positives (FP): 19
True Negatives (TN): 107
False Negatives (FN): 30
Accuracy: 0.77
Error Rate: 0.23
Precision: 0.76
Recall: 0.66
```

9]implement Simple Naïve Bayes Classification on the Iris dataset

1. Import required libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score, precision_score, recall_score
import seaborn as sns
```

2. Load the Titanic dataset

```
df = sns.load_dataset('titanic')
```

3. Select relevant features

```
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
```

```

'fare']]

# 4. Drop rows with missing values
df.dropna(inplace=True)

# 5. Encode categorical features
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# 6. Define features and target
X = df.drop('survived', axis=1)
y = df['survived']

# 7. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 8. Initialize and train Naive Bayes classifier
model = GaussianNB()
model.fit(X_train, y_train)

# 9. Make predictions
y_pred = model.predict(X_test)

# 10. Evaluation
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Not
Survived', 'Survived']))
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")

```

Confusion Matrix:

```

20]
[[ 106
   28]
 [ 28
  106]

```

Classification Report:

	precision	recall	f1-score	support
Not Survived	0.79	0.84	0.82	126
Survived	0.75	0.69	0.72	89
accuracy			0.78	215
macro avg			0.77	215
weighted avg	0.77	0.76	0.77	215
	0.78	0.78		

Accuracy: 0.78 Precision: 0.75 Recall: 0.69

9] Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset

1. Import required libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score
import seaborn as sns
```

2. Load the Titanic dataset

```
df = sns.load_dataset('titanic')
```

3. Select relevant features

```
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
'fare']]
```

4. Drop rows with missing values

```
df.dropna(inplace=True)
```

5. Encode categorical features

```
df['sex'] = df['sex'].map({'male': 0, 'female': 1})
```

6. Define features and target

```
X = df.drop('survived', axis=1)
```

```
y = df['survived']
```

7. Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

8. Initialize and train Naive Bayes classifier

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

9. Make predictions

```
y_pred = model.predict(X_test)
```

10. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:\n", cm)
```

```

# 11. Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 12. Error rate
error_rate = 1 - accuracy
print("Error Rate:", error_rate)

# 13. Precision and Recall (macro averaging)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
print("Precision (macro):", precision)
print("Recall (macro):", recall)

# 14. TP, FP, TN, FN for each class (one-vs-rest)
for i in range(len(cm)):
    TP = cm[i][i]
    FP = sum(cm[:,i]) - TP
    FN = sum(cm[i,:]) - TP
    TN = cm.sum() - (TP + FP + FN)
    print(f"\nClass {i}:")
    print(f"TP: {TP}")
    print(f"FP: {FP}")
    print(f"FN: {FN}")
    print(f"TN: {TN}")

```

Confusion Matrix:

```

[[106   20]
 [ 28   61]]

```

Accuracy: 0.7767441860465116

Error Rate: 0.22325581395348837

Precision (macro): 0.7720655979362447

Recall (macro): 0.7633315498484038

Class 0:

TP: 106

FP: 28

FN: 20

TN: 61

Class 1:

TP: 61

FP: 20

FN: 28

TN: 106

10]calculating Term Frequency and Inverse Document Frequency.


```

from sklearn.datasets import load_iris
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import seaborn as sns

# Step 1: Load the Titanic dataset
df = sns.load_dataset('titanic')

# Step 2: Select relevant columns and drop missing values
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
'fare']]
df.dropna(inplace=True)

# Step 3: Convert rows to text by combining the columns
def row_to_text(row):

    return " ".join([f"{col}_{val}" for col, val in zip(df.columns,
row)])

# Step 4: Apply the function to each row to create the 'documents'
documents = df.apply(row_to_text, axis=1)

# Step 5: Apply TF-IDF Vectorizer
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# Step 6: Convert the result to a DataFrame
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(),
columns=vectorizer.get_feature_names_out())

# Step 7: Show the first few rows
print(tfidf_df.head())

```

	0042	0125	0208	025	0292	0458	05	0542	0708	
075 \	...									
0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	...
1	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	...
2	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	...
3	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	...
4	0.0	0.0	0.0	0.0	0.0	0.0	0.508724	0.0	0.0	...

	sex_female	sex_male	sibsp_0	sibsp_1	sibsp_2	sibsp_3	sibsp_4
0 \	0.000000	0.187286	0.000000	0.303605	0.0	0.0	0.0
1	0.178211	0.000000	0.000000	0.209640	0.0	0.0	0.0

2	0.250804	0.000000	0.177133	0.000000	0.0	0.0	0.0
3	0.231057	0.000000	0.000000	0.271805	0.0	0.0	0.0
4	0.000000	0.185466	0.180507	0.000000	0.0	0.0	0.0

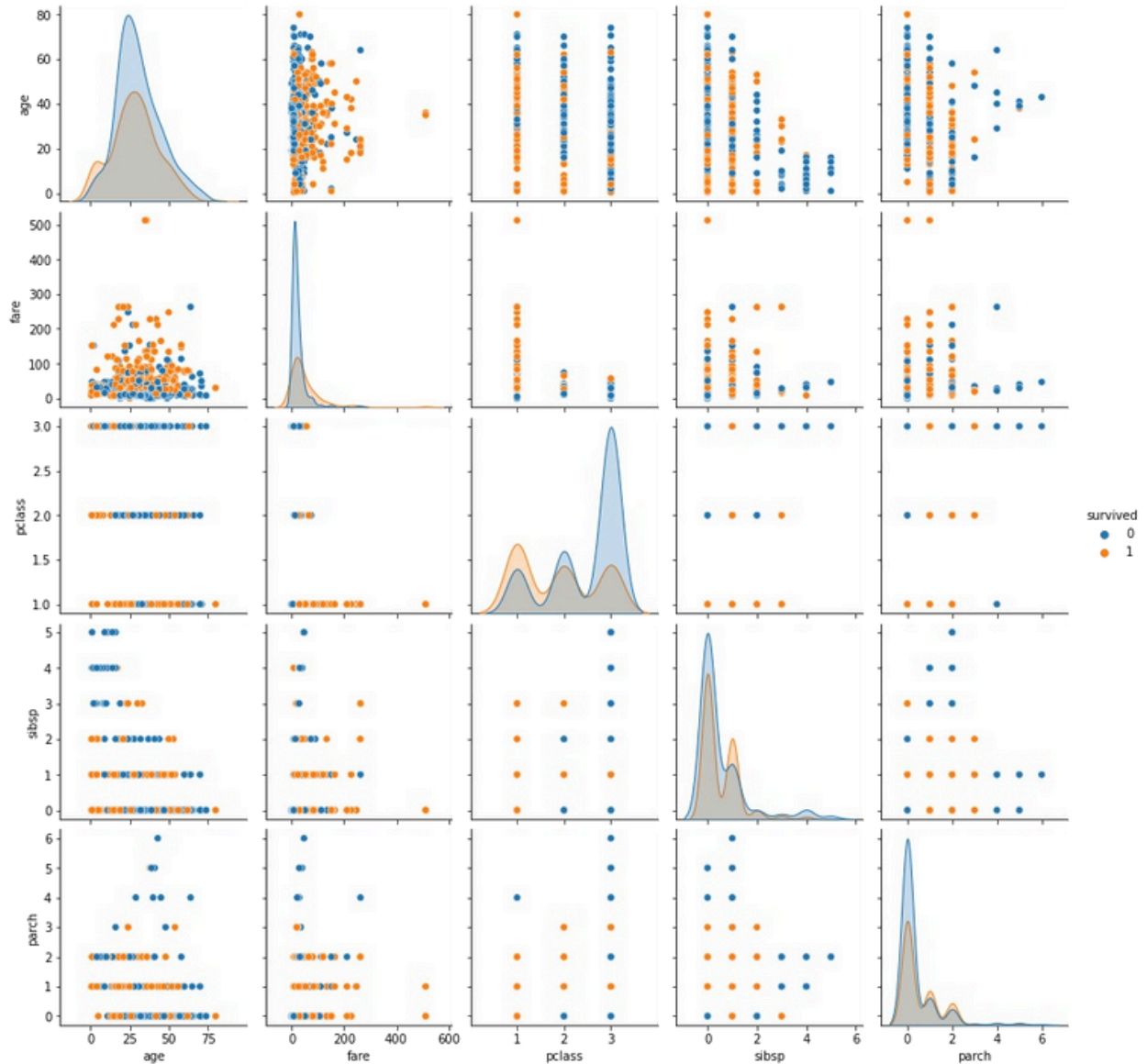
	sibsp_5	survived_0	survived_1
0	0.0	0.195787	0.000000
1	0.0	0.000000	0.168875
2	0.0	0.000000	0.237666
3	0.0	0.000000	0.218953
4	0.0	0.193884	0.000000

[5 rows x 286 columns]

11]Seaborn visualizations like pairplots, boxplots, violin plots, and heatmaps to find patterns in the Iris dataset.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Step 1: Load the Titanic dataset
df = sns.load_dataset('titanic')
df = df.dropna(subset=['age', 'fare', 'pclass', 'sibsp', 'parch',
'survived'])
sns.pairplot(df, hue='survived', vars=['age', 'fare', 'pclass',
'sibsp', 'parch'])
plt.show()
```



12] The code generates box plots for each of the sepal length, sepal width, petal length, and petal width with respect to the species. These box plots visualize the distribution of each feature for each species in the Iris dataset.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
# Load the Titanic dataset
```

```
df = sns.load_dataset('titanic')
```

```
# Step 1: Drop rows with missing values for simplicity
```

```
df = df.dropna(subset=['age', 'fare', 'sibsp', 'parch', 'survived'])
```

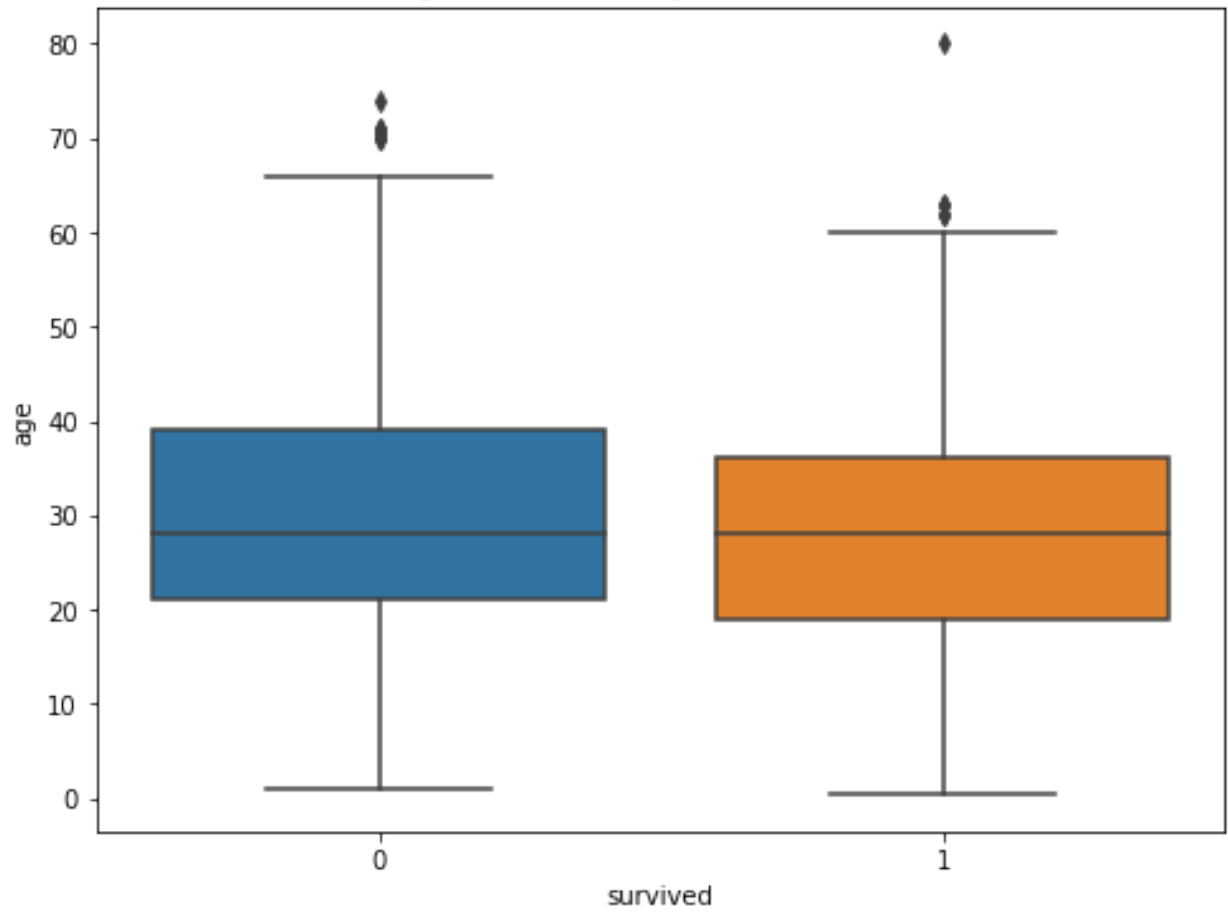
```
# Create a box plot for Age distribution by Survived status
plt.figure(figsize=(8, 6))
sns.boxplot(x='survived', y='age', data=df)
plt.title('Age Distribution by Survival Status')
plt.show()

# Create a box plot for Fare distribution by Survived status
plt.figure(figsize=(8, 6))
sns.boxplot(x='survived', y='fare', data=df)
plt.title('Fare Distribution by Survival Status')
plt.show()

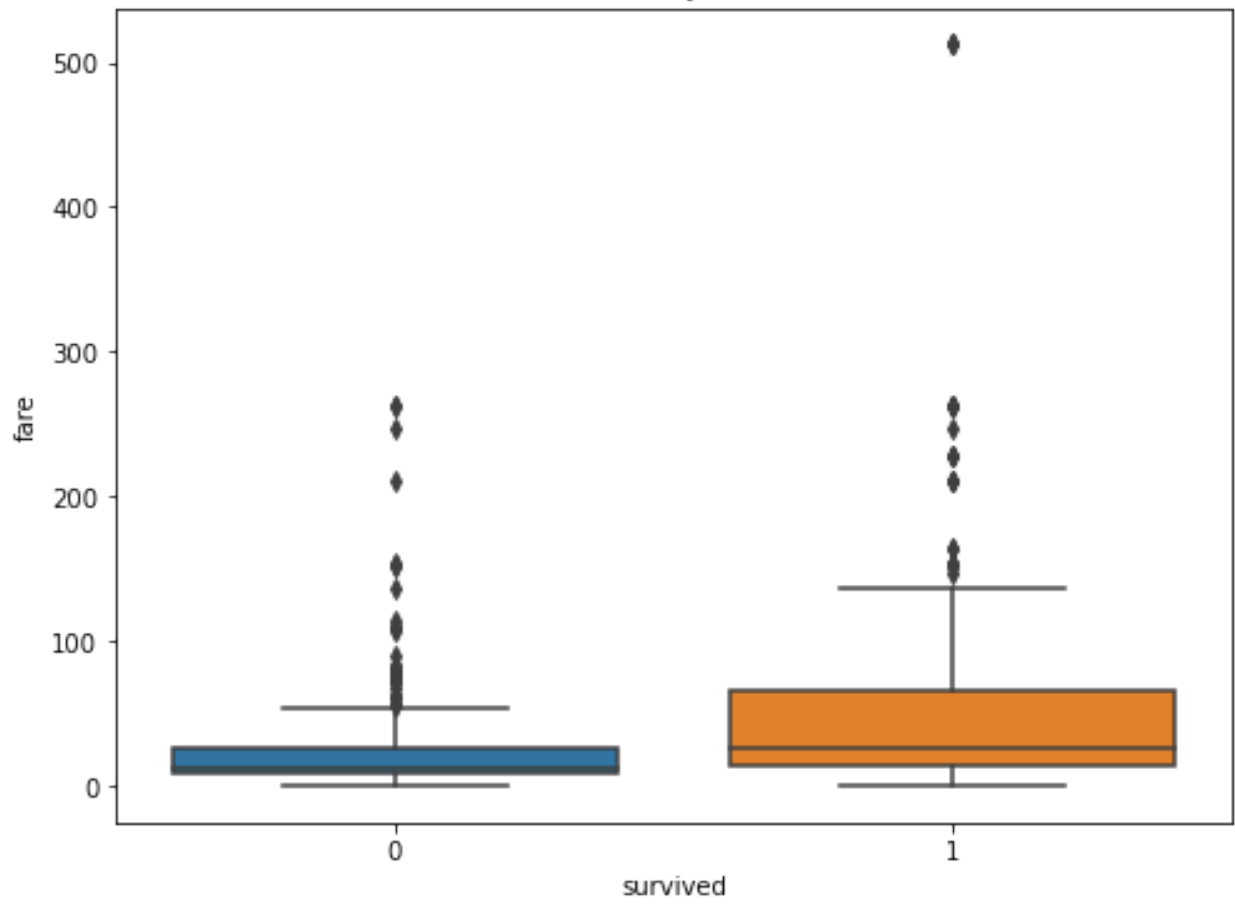
# Create a box plot for SibSp (siblings/spouses) distribution by Survived status
plt.figure(figsize=(8, 6))
sns.boxplot(x='survived', y='sibsp', data=df)
plt.title('Siblings/Spouses Distribution by Survival Status')
plt.show()

# Create a box plot for Parch (parents/children) distribution by Survived status
plt.figure(figsize=(8, 6))
sns.boxplot(x='survived', y='parch', data=df)
plt.title('Parents/Children Distribution by Survival Status')
plt.show()
```

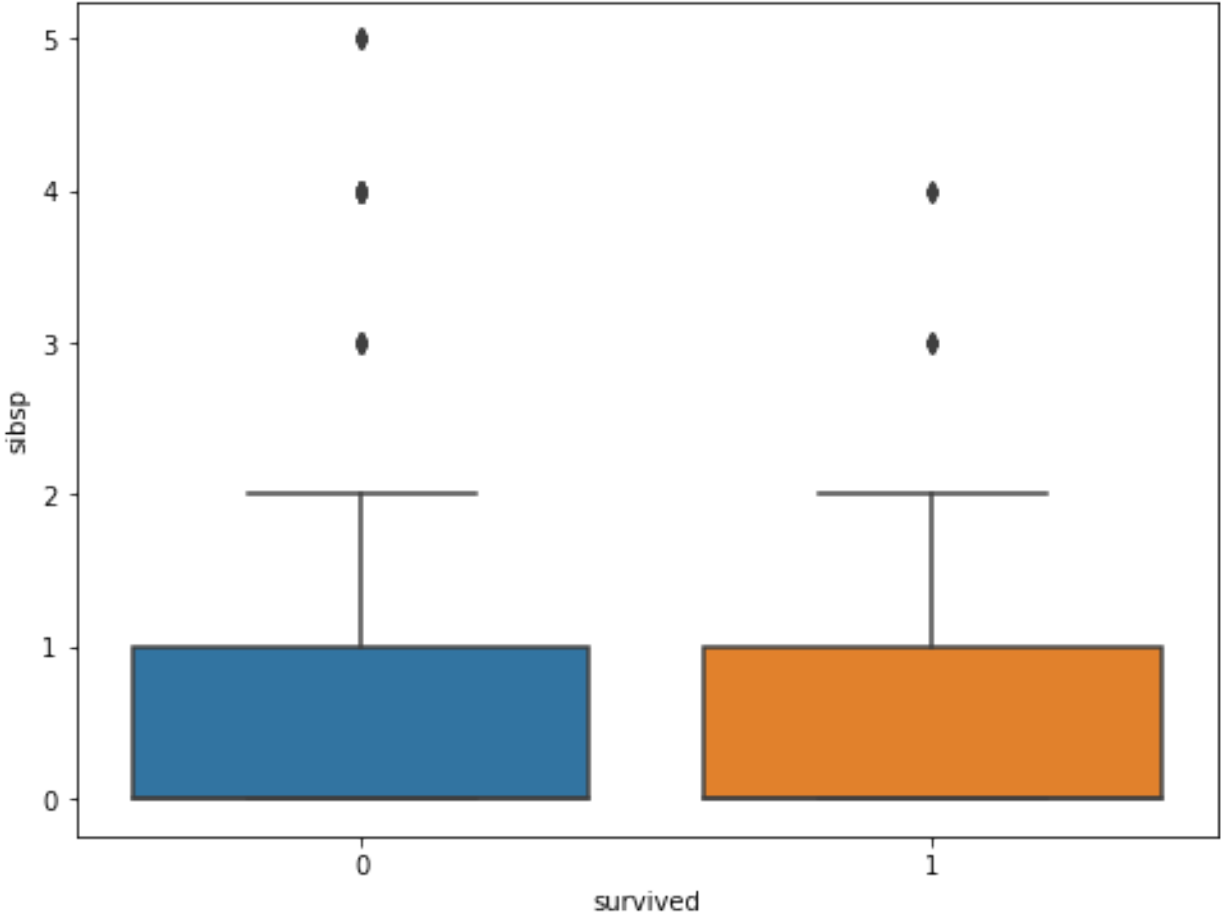
Age Distribution by Survival Status

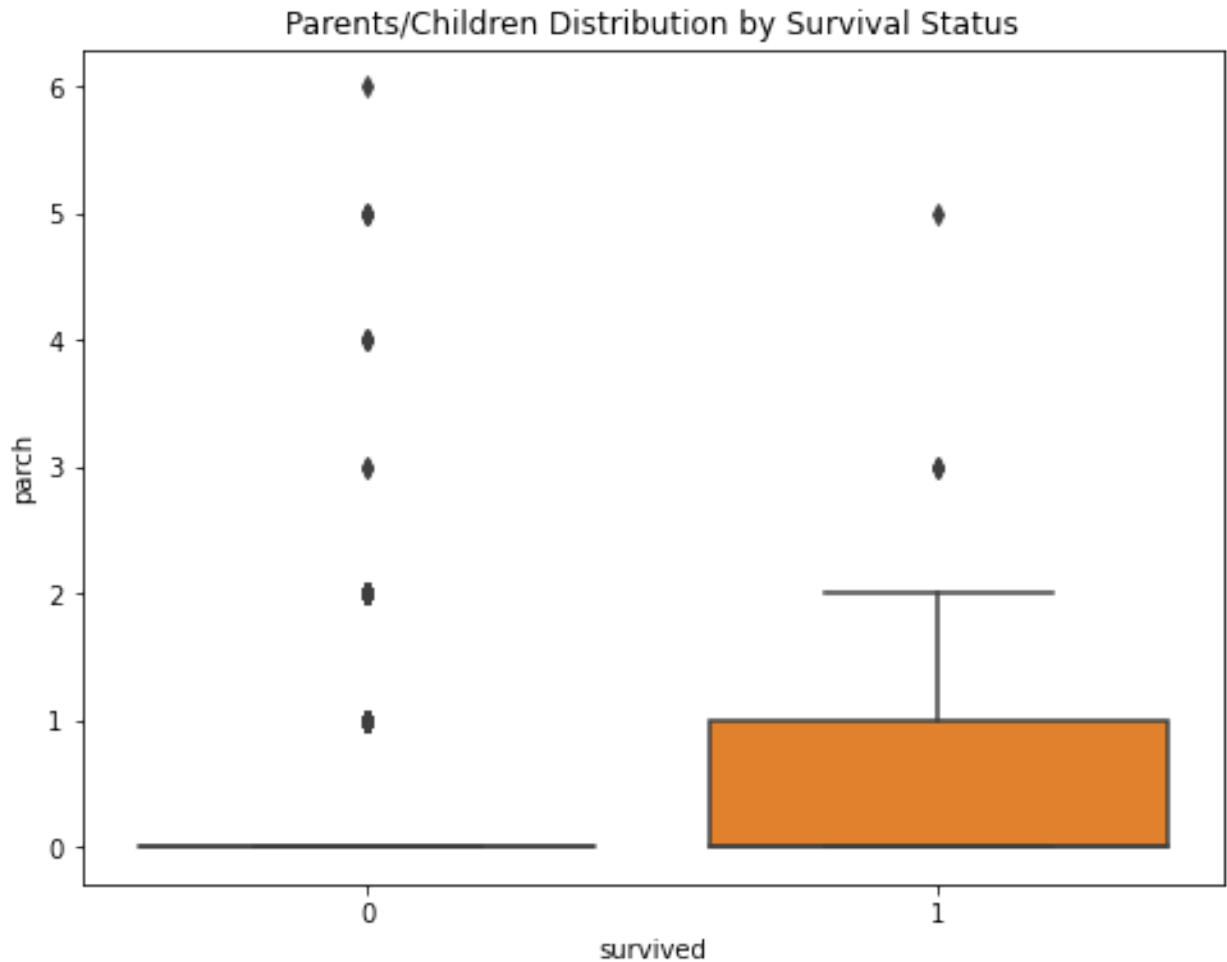


Fare Distribution by Survival Status



Siblings/Spouses Distribution by Survival Status





12]. List down the features **and** their types (e.g., numeric, nominal) available **in** the dataset.

2. Create a histogram **for** each feature **in** the dataset to illustrate the feature distributions.

3. Create a boxplot **for** each feature **in** the dataset.

4. Compare distributions **and** identify outliers.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Load the Titanic dataset from seaborn
```

```
df = sns.load_dataset('titanic')
```

```
# Drop rows with missing values for simplicity (you can handle missing data differently)
```

```
df.dropna(subset=['age', 'fare', 'sibsp', 'parch', 'survived'],  
inplace=True)
```

```
# 1. List down the features and their types
```



```

features = df.columns
feature_types = {
    'pclass': 'nominal', 'sex':
    'nominal', 'age': 'numeric',
    'sibsp': 'numeric', 'parch':
    'numeric', 'fare': 'numeric',
    'embarked': 'nominal',
    'survived': 'nominal',
    'class': 'nominal', 'who':
    'nominal', 'alone':
    'nominal', 'deck':
    'nominal', 'embark_town':
    'nominal', 'alive':
    'nominal', 'alone':
    'nominal'
}

print("Features and their types:")
for feature in features:
    if feature in feature_types:
        print(f"- {feature}: {feature_types[feature]}")

# 2. Create histograms for each numerical feature
df[['age', 'sibsp', 'parch', 'fare']].hist(figsize=(12, 8))
plt.suptitle("Histograms of Each Feature", y=0.92)
plt.show()

# 3. Create a boxplot for each numerical feature by 'survived'
plt.figure(figsize=(12, 8))
for i, feature in enumerate(['age', 'sibsp', 'parch', 'fare']):

    plt.subplot(2, 2, i + 1)
    sns.boxplot(x='survived', y=feature, data=df)
    plt.title(f'Boxplot of {feature} by Survival Status')

plt.tight_layout()
plt.subplots_adjust(top=0.9)
plt.suptitle("Boxplots of Features by Survival Status", fontsize=16)
plt.show()

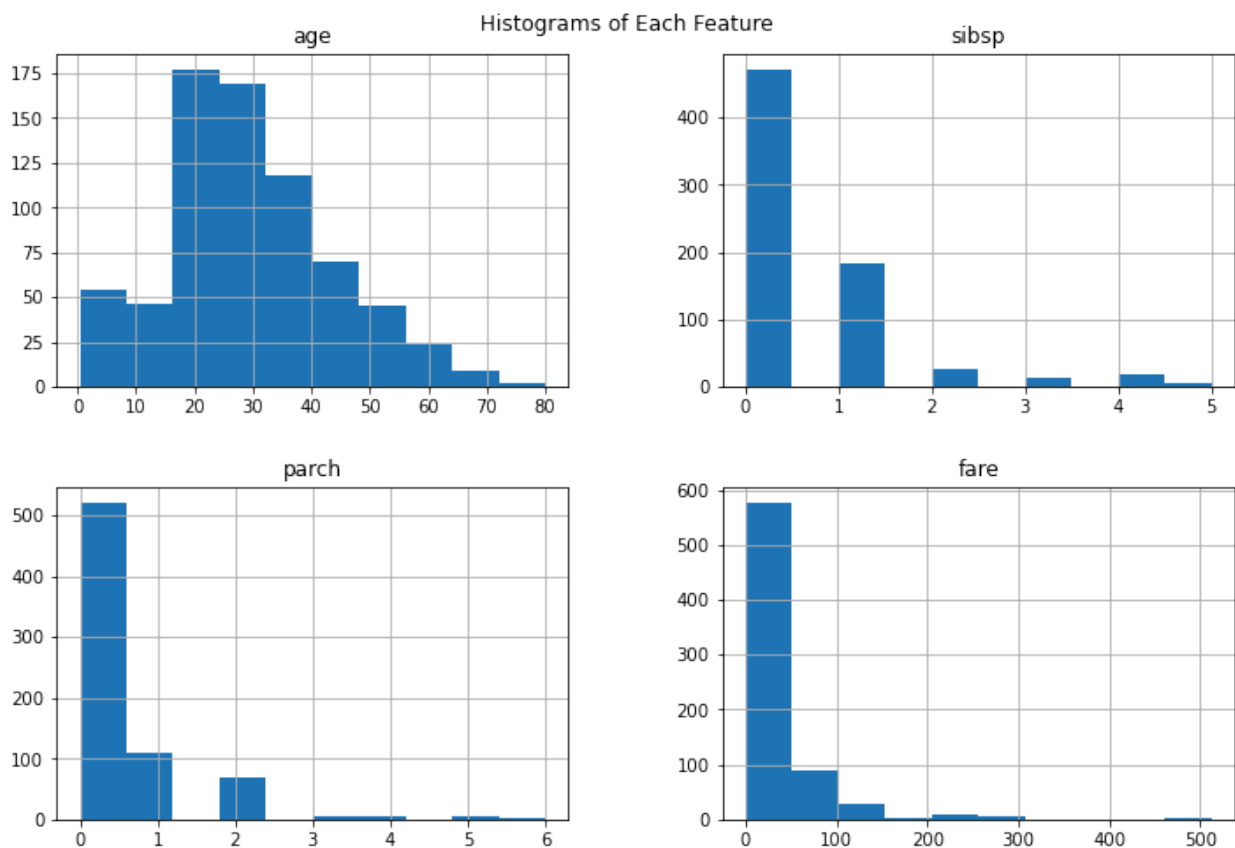
# 4. Compare distributions and identify outliers
print("\nObservations and Outlier Detection:")
print("- Age: The age distribution shows a peak in the younger ages. There are potential outliers in the older age group.")
print("- Siblings/Spouses (SibSp): The distribution is mostly skewed with several people having 0 siblings/spouses aboard. No obvious outliers.")
print("- Parents/Children (Parch): Similar to SibSp, the distribution is mostly skewed, with several people having 0 parents/children")

```

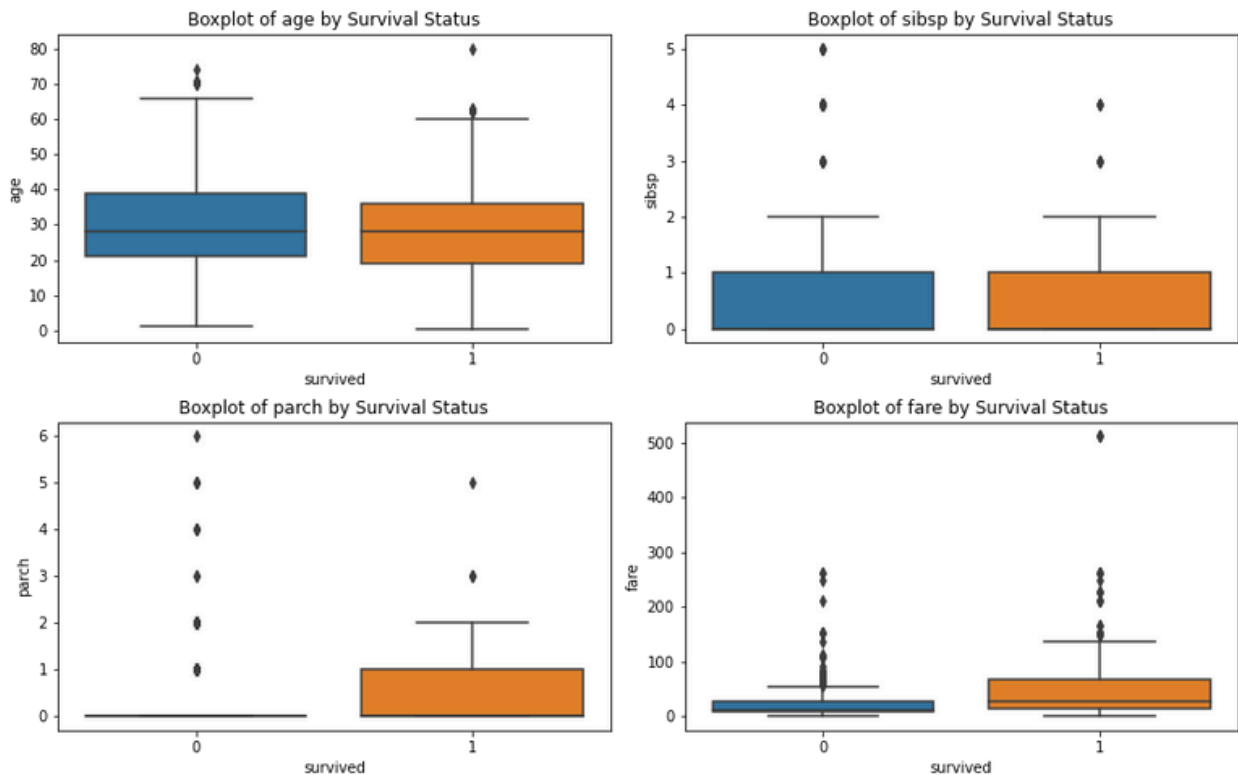
aboard. No obvious outliers.") `print("- Fare: The distribution has a wide spread, with a significant number of passengers having low fare values. Some high-fare values may indicate outliers.")`

Features and their types:

- survived: nominal
- pclass: nominal
- sex: nominal
- age: numeric
- sibsp: numeric
- parch: numeric
- fare: numeric
- embarked: nominal
- class: nominal
- who: nominal
- deck: nominal
- embark_town: nominal
- alive: nominal
- alone: nominal



Boxplots of Features by Survival Status



Observations and Outlier Detection:

- Age: The age distribution shows a peak in the younger ages. There are potential outliers in the older age group.
- Siblings/Spouses (SibSp): The distribution is mostly skewed with several people having 0 siblings/spouses aboard. No obvious outliers.
- Parents/Children (Parch): Similar to SibSp, the distribution is mostly skewed, with several people having 0 parents/children aboard. No obvious outliers.
- Fare: The distribution has a wide spread, with a significant number of passengers having low fare values. Some high-fare values may indicate outliers.